
Eduardo Gomes Martins e Lucas de Lima Bergami

Descoberta de Conhecimento com a Base de Dados do Github

1 Introdução

A descoberta de conhecimento em bases de dados, conhecida pela sigla KDD (Knowledge Discovery in Databases), compreende um processo sistemático de extração de informações úteis a partir de grandes volumes de dados brutos. Esse processo envolve etapas que vão desde a coleta e o pré-processamento dos dados até a aplicação de técnicas de mineração e a interpretação dos resultados obtidos. O foco do presente trabalho não reside na descoberta de conhecimento em si, mas na execução metodológica dessas etapas, tratando-as como objeto central de estudo.

Para a realização deste trabalho, foram coletados dados da plataforma GitHub por meio de sua API pública REST. O GitHub constitui uma rede social de desenvolvedores amplamente utilizada, na qual usuários compartilham repositórios de código, colaboram em projetos e interagem por meio de contribuições, pull requests e comentários. A coleta foi realizada a partir de um conjunto de usuários-semente (Linus Torvalds, Andrej Karpathy, TensorFlow, PyTorch e Hugging Face) expandindo-se para seus seguidores de forma a compor uma base diversificada de perfis de desenvolvedores.

Foram coletados dois conjuntos de dados distintos: um referente aos usuários, contendo atributos de perfil social, atividade recente, linguagens de programação utilizadas e métricas de engajamento; e outro referente aos repositórios, contendo métricas de popularidade, composição tecnológica, atividade de colaboração e características estruturais. A partir desses dois conjuntos, foi construído um dataset relacional, no qual cada instância representa um par usuário-repositório enriquecido com features de compatibilidade tecnológica, diferenças de escala e métricas relacionais derivadas.

O problema modelado é de classificação supervisionada binária: dado um par usuário-repositório, o objetivo é prever se o usuário é ou não um contribuidor daquele repositório. Os pares positivos foram identificados diretamente pela propriedade dos repositórios, enquanto os pares negativos foram gerados por amostragem estratificada, priorizando negativos difíceis — isto é, pares em que o usuário compartilha ao menos uma linguagem com o repositório, tornando a tarefa de discriminação mais desafiadora e o modelo mais robusto.

Nas seções seguintes, são descritos os atributos investigados, as estatísticas básicas do conjunto de dados, as distribuições observadas, o processo de seleção e normalização de features, a redução de dimensionalidade aplicada e, por fim, as técnicas de mineração de dados adotadas (Random Forest e Regressão Logística) com análise comparativa e avaliação dos resultados obtidos.

2 Coleta de Dados

A construção do dataset foi realizada utilizando a API REST do GitHub, permitindo a extração automática de informações públicas de usuários e repositórios. O processo foi dividido

em duas etapas principais: coleta de usuários e coleta de repositórios.

O objetivo da coleta foi construir uma base de dados capaz de representar características comportamentais, sociais e técnicas de usuários do GitHub, bem como atributos estruturais e de popularidade dos repositórios. Essas informações foram posteriormente utilizadas para construir um dataset relacional voltado para tarefas de classificação envolvendo colaboração entre usuários e projetos.

2.1 Coleta de Repositórios

Após a coleta dos usuários, foi realizada a extração dos repositórios associados a cada perfil.

Para aumentar a qualidade do dataset, foram aplicados filtros para remover forks, repositórios sem linguagem principal definida, projetos muito pequenos e repositórios com baixa quantidade de estrelas.

Os repositórios válidos foram então ordenados por relevância, considerando principalmente o número de estrelas e forks.

Para cada repositório foram extraídas informações relacionadas à popularidade, colaboração, atividade, linguagens utilizadas, tópicos do projeto e características estruturais do repositório.

Além disso, foram realizadas consultas adicionais à API do GitHub para coletar informações mais detalhadas, como as linguagens utilizadas no projeto, os tópicos associados ao repositório e a quantidade de contribuidores.

2.2 Tratamento de Limite da API

A API do GitHub possui limite de requisições por hora. Para evitar perda de progresso durante a coleta, o sistema implementa mecanismos de checkpoint.

Os dados coletados são salvos periodicamente em arquivos CSV temporários, permitindo que a execução continue posteriormente caso o limite da API seja atingido ou ocorra alguma falha durante a coleta.

Além disso, o sistema monitora continuamente o número de requisições restantes utilizando os cabeçalhos retornados pela própria API.

3 Descrição dos Atributos Coletados

3.1 Atributos dos Repositorios

Atributo	Descrição
repo_name	Nome completo do repositório no GitHub.
repo_id	Identificador único do repositório na plataforma.
owner	Usuário proprietário do repositório.
repo_stars	Quantidade de estrelas recebidas pelo repositório.
repo_forks	Quantidade de forks do repositório.
repo_watchers	Quantidade de usuários acompanhando o repositório.
repo_open_issues	Número de issues abertas.
repo_size	Tamanho do repositório em KB.
main_language	Linguagem principal identificada pelo GitHub.
dominant_language	Linguagem dominante considerando volume de código.
dominant_language_ratio	Proporção da linguagem dominante em relação ao total de código.
languages	Lista de linguagens utilizadas no repositório.
num_languages	Quantidade de linguagens utilizadas.
topics	Lista de tópicos associados ao repositório.
num_topics	Quantidade de tópicos associados.
contributors_count	Número de contribuidores do repositório.
has_description	Indica se o repositório possui descrição.
description_length	Tamanho da descrição do repositório.
repo_age_days	Idade do repositório em dias desde sua criação.
days_since_update	Quantidade de dias desde a última atualização.
has_wiki	Indica se o repositório possui wiki habilitada.
has_pages	Indica se o GitHub Pages está habilitado.
has_projects	Indica se projetos do GitHub estão habilitados.
has_downloads	Indica se downloads estão habilitados.
has_issues	Indica se o sistema de issues está habilitado.
stars_per_contributor	Média de estrelas por contribuidor.
forks_per_contributor	Média de forks por contribuidor.
html_url	URL pública do repositório.
api_url	URL da API do repositório.

Tabela 1: Descrição dos atributos coletados dos repositórios GitHub

3.2 Atributos dos Usuários

Atributo	Descrição
username	Nome de usuário único na plataforma GitHub.
account_type	Tipo da conta, podendo ser usuário individual (<i>User</i>) ou organização (<i>Organization</i>).
followers	Quantidade de seguidores do usuário na plataforma.
following	Quantidade de usuários que o perfil segue.
follower_following_ratio	Razão entre seguidores e seguidos, indicando o nível de influência relativa do usuário.
public_repos	Quantidade de repositórios públicos pertencentes ao usuário.
public_gists	Quantidade de gists públicos criados pelo usuário.
total_stars_received	Soma total de estrelas recebidas em todos os repositórios do usuário.
total_forks_owned	Soma total de forks realizados nos repositórios do usuário.
total_watchers	Soma total de watchers nos repositórios do usuário.
total_open_issues	Soma total de issues abertas nos repositórios do usuário.
languages	Lista de linguagens de programação utilizadas nos repositórios do usuário, separadas por pipe.
num_languages	Quantidade de linguagens de programação distintas utilizadas pelo usuário.
dominant_language	Linguagem de programação com maior volume de código entre os repositórios analisados.
bio_length	Tamanho em caracteres da biografia do perfil do usuário.
has_company	Indica se o usuário possui empresa associada ao perfil.
has_blog	Indica se o usuário possui blog ou site pessoal cadastrado.
has_twitter	Indica se o usuário possui conta no Twitter/X vinculada ao perfil.
has_readme_profile	Indica se o usuário possui repositório especial de perfil com README.
location	Localização geográfica informada pelo usuário em seu perfil.
hireable	Indica se o usuário sinalizou estar disponível para contratação.
is_site_admin	Indica se o usuário possui privilégios administrativos na plataforma.
push_events	Quantidade de eventos de push realizados recentemente pelo usuário.
pr_events	Quantidade de eventos de pull request realizados recentemente.
issue_events	Quantidade de eventos relacionados a issues realizados recentemente.
fork_events	Quantidade de eventos de fork realizados recentemente pelo usuário.
watch_events	Quantidade de eventos de watch realizados recentemente.
create_events	Quantidade de eventos de criação de branch ou repositório realizados recentemente.
comment_events	Quantidade de eventos de comentário realizados recentemente.
total_recent_events	Quantidade total de eventos públicos recentes do usuário, abrangendo todos os tipos.
starred_repos_count	Quantidade de repositórios marcados com estrela pelo usuário.
watching_repos_count	Quantidade de repositórios que o usuário está acompanhando.
account_age_days	Idade da conta em dias, calculada a partir da data de criação do perfil.
last_updated	Data e hora da última atualização registrada no perfil do usuário.

Tabela 2: Descrição dos atributos coletados dos usuários GitHub

4 Construção da Tabela Correlacionada

Após a coleta dos dados de usuários e repositórios, foi construída uma tabela correlacionada com o objetivo de representar relações entre desenvolvedores e projetos do GitHub.

Inicialmente, foram gerados pares positivos utilizando os repositórios cujo proprietário estava presente no conjunto de usuários coletados. Nesse caso, cada par usuário–repositório recebeu o rótulo *label* = 1, indicando uma relação real entre o usuário e o projeto.

Em seguida, foram gerados pares negativos balanceados, onde usuários foram associados aleatoriamente a repositórios que não lhes pertenciam. Esses pares receberam o rótulo *label* = 0. Para tornar o problema mais desafiador e evitar exemplos triviais, parte dos negativos foi gerada utilizando usuários e repositórios que compartilhavam ao menos uma linguagem de programação em comum, criando assim exemplos negativos mais difíceis.

Após a criação dos pares, foi realizado o processo de junção (*merge*) entre os datasets de

usuários e repositórios, utilizando os identificadores de usuário e nome do repositório. Isso permitiu combinar informações dos dois conjuntos em uma única estrutura tabular.

Com os dados integrados, foram extraídas diversas features relacionais entre usuários e repositórios. Entre elas estão:

- correspondência entre linguagens principais;
- quantidade de linguagens em comum;
- similaridade de Jaccard entre conjuntos de linguagens;
- diferença entre métricas de popularidade;
- razão entre métricas sociais e de contribuição;
- indicadores de atividade e experiência.

Também foram mantidas features individuais tanto dos usuários quanto dos repositórios, permitindo que o modelo utilizasse simultaneamente informações específicas e relacionais.

Por fim, o dataset passou por uma etapa de limpeza, incluindo tratamento de valores ausentes, substituição de valores infinitos e normalização posterior durante o treinamento dos modelos de aprendizado de máquina.

5 Descrição das Features do Dataset Relacional

5.1 Identificação e Rótulo

Feature	Descrição
label	Variável alvo do problema de classificação. Valor 1 indica que o usuário contribui para o repositório; valor 0 indica ausência de contribuição observada.
username	Nome de usuário no GitHub utilizado para identificar o desenvolvedor analisado.
repo_name	Nome completo do repositório analisado (owner/repository).

Tabela 3: Features de identificação e rótulo

5.2 Compatibilidade Tecnológica

Feature	Descrição
dominant_language_match	Indica se a linguagem dominante do usuário é igual à linguagem dominante do repositório.
main_language_match	Indica se a linguagem principal do repositório aparece entre as linguagens utilizadas pelo usuário.
language_overlap	Quantidade de linguagens em comum entre usuário e repositório.
language_jaccard	Similaridade de Jaccard entre os conjuntos de linguagens do usuário e do repositório. Mede o grau de compatibilidade tecnológica entre ambos.

Tabela 4: Features de compatibilidade tecnológica

5.3 Diferenças de Popularidade e Escala

Feature	Descrição
stars_gap	Diferença absoluta entre o total de estrelas recebidas pelos repositórios do usuário e a quantidade de estrelas do repositório analisado.
forks_gap	Diferença absoluta entre o total de forks associados ao usuário e a quantidade de forks do repositório.
followers_gap	Diferença absoluta entre o número de seguidores do usuário e a quantidade de watchers do repositório.
age_gap	Diferença absoluta entre a idade da conta do usuário e a idade do repositório.

Tabela 5: Features de diferenças estruturais

5.4 Razões e Métricas Relativas

Feature	Descrição
user_repo_popularity_ratio	Razão entre o total de estrelas associadas ao usuário e o número de estrelas do repositório. Mede compatibilidade de popularidade.
contributor_popularity_ratio	Razão entre o número de seguidores do usuário e a quantidade de contribuidores do repositório.

Tabela 6: Features de razões e proporções

5.5 Features Booleanas

Feature	Descrição
experienced_user	Indica se o usuário possui experiência relevante na plataforma, considerando um número mínimo de repositórios públicos.
popular_repo	Indica se o repositório pode ser considerado popular com base na quantidade de estrelas recebidas.
active_user	Indica se o usuário apresentou atividade recente na plataforma.
active_repo	Indica se o repositório foi atualizado recentemente.
has_social_presence	Indica se o usuário possui presença social adicional, como blog pessoal ou conta no Twitter/X.

Tabela 7: Features booleanas

5.6 Features do Usuário

Feature	Descrição
followers	Número de seguidores do usuário no GitHub.
following	Número de usuários seguidos pelo usuário analisado.
public_repos	Quantidade de repositórios públicos pertencentes ao usuário.
total_stars_received	Soma total de estrelas recebidas pelos repositórios do usuário.
num_languages_user	Quantidade de linguagens de programação utilizadas pelo usuário.
account_age_days	Idade da conta do usuário em dias.
total_recent_events	Quantidade total de eventos recentes associados ao usuário, como commits, pull requests e comentários.

Tabela 8: Features do usuário

5.7 Features do Repositório

Feature	Descrição
repo_stars	Quantidade de estrelas do repositório.
repo_forks	Quantidade de forks do repositório.
repo_watchers	Número de usuários observando o repositório.
repo_size	Tamanho do repositório.
contributors_count	Quantidade de contribuidores associados ao repositório.
num_languages_repo	Quantidade de linguagens utilizadas no repositório.
repo_age_days	Idade do repositório em dias.
days_since_update	Quantidade de dias desde a última atualização do repositório.

Tabela 9: Features do repositório

5.8 Estatísticas Descritivas da Tabela Correlacionada

A Tabela 10 apresenta estatísticas descritivas das principais variáveis utilizadas no dataset correlacionado. Foram calculados os valores mínimo, máximo, média, mediana e desvio padrão para cada atributo numérico.

Os resultados mostram uma grande variabilidade entre os usuários e repositórios coletados. Algumas variáveis, como *repo_stars*, *repo_forks* e *followers*, apresentaram elevados desvios padrão, indicando a presença de usuários e projetos muito populares em comparação aos demais.

As variáveis relacionais, como *language_overlap* e *language_jaccard*, apresentaram médias relativamente baixas, sugerindo que muitos pares usuário-repositório possuem pouca similaridade direta entre linguagens utilizadas.

Também é possível observar que a maior parte dos usuários possui atividade recente, evidenciado pelas variáveis booleanas *active_user* e *experienced_user*, cujas médias ficaram próximas de 1.

Feature	Min	Max	Mean	Median	Std
label	0.0	1.0	0.500	0.500	0.502
dominant_language_match	0.0	1.0	0.156	0.000	0.365
main_language_match	0.0	1.0	0.359	0.000	0.482
language_overlap	0.0	6.0	1.094	1.000	1.146
language_jaccard	0.0	0.75	0.147	0.106	0.170
stars_gap	2.0	1292.0	144.031	39.000	240.053
forks_gap	0.0	1763.0	37.859	8.000	159.008
followers_gap	2.0	3541.0	283.648	105.500	558.755
age_gap	0.0	6608.0	1912.461	1791.500	1669.204
user_repo_popularity_ratio	0.0	1.008	0.046	0.000	0.141
contributor_popularity_ratio	0.0	3563.0	151.011	42.250	383.060
experienced_user	0.0	1.0	0.922	1.000	0.269
popular_repo	0.0	1.0	0.313	0.000	0.465
active_user	0.0	1.0	0.953	1.000	0.212
active_repo	0.0	1.0	0.602	1.000	0.492
has_social_presence	0.0	1.0	0.750	1.000	0.435
followers	0.0	3563.0	283.695	115.000	602.275
following	0.0	7389.0	477.133	85.000	1236.762
public_repos	3.0	1963.0	139.484	60.000	244.785
total_stars_received	0.0	243.0	3.078	0.000	21.937
num_languages_user	0.0	55.0	7.086	5.000	7.728
account_age_days	1760.0	6685.0	6501.742	6650.000	457.993
total_recent_events	0.0	100.0	35.469	13.000	40.663
repo_stars	5.0	1292.0	147.078	40.000	240.679
repo_forks	0.0	1763.0	38.211	8.000	158.978
repo_watchers	5.0	1292.0	147.078	40.000	240.679
repo_size	69.0	529940.0	26546.055	833.000	85452.823
contributors_count	0.0	52.0	5.555	2.000	10.438
num_languages_repo	1.0	10.0	2.906	2.000	2.236
repo_age_days	60.0	6686.0	4680.438	4726.000	1713.221
days_since_update	0.0	3511.0	530.148	137.000	746.415

Tabela 10: Estatísticas descritivas das features do dataset correlacionado

6 Técnicas de Mineração de Dados

6.1 Random Forest

O Random Forest é um algoritmo de aprendizado de máquina supervisionado baseado no conceito de *ensemble learning*, ou seja, combina múltiplos modelos mais simples para produzir uma predição final mais robusta. No caso do Random Forest, o ensemble é composto por uma coleção de árvores de decisão, cada uma treinada de forma independente sobre uma amostra aleatória do conjunto de treinamento, técnica conhecida como *bootstrap aggregating* (bagging).

Cada árvore de decisão aprende a classificar as instâncias particionando recursivamente o espaço de features com base em critérios de pureza, como o índice de Gini ou a entropia. A aleatoriedade é introduzida também na seleção de features: em cada nó da árvore, apenas um subconjunto aleatório de atributos é considerado para a divisão, o que reduz a correlação entre as árvores e aumenta a diversidade do ensemble.

A predição final é obtida por votação majoritária entre todas as árvores, o que torna o modelo menos suscetível a overfitting em comparação com uma única árvore de decisão. Além disso, o Random Forest fornece naturalmente uma medida de importância de cada feature, calculada com base na redução média de impureza promovida por cada atributo ao longo de todas as árvores, informação utilizada neste trabalho para identificar os atributos mais relevantes para a

classificação.

Neste trabalho, o modelo foi configurado com 300 estimadores, profundidade máxima de 15 e pesos de classe balanceados (`class_weight=balanced`), estratégia adotada para compensar o desbalanceamento entre pares contribuidores e não-contribuidores.

6.2 Regressão Logística

A Regressão Logística é um algoritmo de classificação linear que modela a probabilidade de uma instância pertencer a uma determinada classe. Apesar do nome, trata-se de um modelo de classificação, não de regressão. O nome deriva do uso da função logística (ou sigmoide), que transforma uma combinação linear das features de entrada em um valor de probabilidade entre 0 e 1.

Formalmente, dado um vetor de features $\mathbf{x}$, o modelo estima a probabilidade da classe positiva como:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (6.1)$$

onde $\mathbf{w}$ é o vetor de pesos aprendidos e b é o termo de viés. O treinamento consiste em encontrar os valores de $\mathbf{w}$ e b que minimizam a função de perda de entropia cruzada (*cross-entropy loss*) sobre o conjunto de treinamento, geralmente por meio de métodos de otimização baseados em gradiente.

Por ser um modelo linear, a Regressão Logística assume que a fronteira de decisão entre as classes pode ser aproximada por um hiperplano no espaço de features. Essa suposição torna o modelo interpretável e computacionalmente eficiente, porém potencialmente menos expressivo do que modelos não-lineares em problemas de maior complexidade. Neste trabalho, o modelo foi configurado com pesos de classe balanceados e limite de iterações de 5000 para garantir a convergência do otimizador.

7 Configuração dos Modelos e Estratégias de Generalização

7.1 Variação de Hiperparâmetros

A fim de reduzir o bias e a variância dos resultados, ambos os modelos foram avaliados sob diferentes configurações de hiperparâmetros. Para o **Random Forest**, as principais configurações avaliadas foram:

- **n_estimators**: número de árvores no ensemble. Valores maiores reduzem a variância, mas aumentam o custo computacional. Foram testados valores de 100, 200 e 300 árvores, sendo 300 o valor que apresentou maior estabilidade nas métricas.
- **max_depth**: profundidade máxima de cada árvore. Árvores muito profundas tendem a memorizar o conjunto de treinamento (overfitting), enquanto árvores rasas podem ser excessivamente simples (underfitting). Foram testadas profundidades de 5, 10 e 15, sendo 15 o valor adotado no modelo final.

- **class_weight**: configurado como `balanced` para compensar o desbalanceamento entre as classes, atribuindo automaticamente pesos maiores à classe minoritária durante o treinamento.

Para a **Regressão Logística**, as configurações avaliadas incluíram:

- **C**: parâmetro de regularização inversa. Valores menores de *C* aplicam maior regularização, reduzindo a variância ao custo de maior bias. Foram testados valores de 0.01, 0.1, 1.0 e 10.0.
- **max_iter**: número máximo de iterações do otimizador, configurado em 5000 para garantir a convergência do modelo sobre o conjunto de dados.
- **class_weight**: também configurado como `balanced`, pela mesma razão aplicada ao Random Forest.

7.2 Estratégias para Minimizar o Overfitting

O overfitting ocorre quando um modelo aprende os padrões específicos do conjunto de treinamento de forma excessivamente detalhada, perdendo a capacidade de generalizar para novos dados. Neste trabalho, as seguintes estratégias foram adotadas para mitigar esse fenômeno:

Divisão treino/teste estratificada. O conjunto de dados foi dividido em 80% para treinamento e 20% para teste, utilizando amostragem estratificada (`stratify=y`). Essa abordagem garante que a proporção entre as classes seja preservada em ambas as partições, evitando que o modelo seja avaliado sobre uma distribuição diferente da que foi treinado.

Limitação da profundidade das árvores. No Random Forest, a profundidade máxima das árvores foi limitada a 15 níveis. Sem essa restrição, cada árvore poderia crescer até memorizar completamente os dados de treinamento, resultando em alta variância. A limitação força as árvores a aprenderem representações mais generalizáveis.

Regularização na Regressão Logística. A Regressão Logística utiliza regularização L2 por padrão, penalizando pesos muito grandes no vetor de parâmetros. Isso impede que o modelo atribua importância excessiva a features ruidosas ou pouco representativas, reduzindo o risco de overfitting.

Ensemble de árvores. A própria natureza do Random Forest é uma estratégia anti-overfitting: ao combinar centenas de árvores treinadas sobre amostras e subconjuntos de features diferentes, os erros individuais de cada árvore tendem a se cancelar, resultando em um modelo mais estável e menos sensível a outliers e ruídos no conjunto de treinamento.

Normalização dos dados. Todos os atributos numéricos foram normalizados com `StandardScaler` antes do treinamento, centralizando os valores em torno de zero com desvio-padrão unitário. Essa etapa é especialmente importante para a Regressão Logística, cujo desempenho é sensível à escala das features, e contribui para uma convergência mais estável do otimizador.

Negativos difíceis. Durante a construção do dataset, 70% dos pares negativos foram gerados como negativos difíceis — pares em que o usuário compartilha ao menos uma linguagem com o repositório, mas não é contribuidor. Essa estratégia força o modelo a aprender discriminações mais sutis, reduzindo o risco de que ele aprenda apenas padrões superficiais e triviais.

7.3 Avaliação dos Resultados

A avaliação dos modelos foi realizada sobre o conjunto de teste, utilizando um conjunto abrangente de métricas para capturar diferentes aspectos do desempenho do classificador:

Acurácia mede a proporção de predições corretas sobre o total de instâncias. Embora seja uma métrica intuitiva, pode ser enganosa em cenários com classes desbalanceadas, razão pela qual foi complementada pelas demais métricas.

Precisão (*precision*) indica, dentre as instâncias classificadas como positivas pelo modelo, qual proporção é de fato positiva. Uma precisão alta significa poucos falsos positivos.

Revocação (*recall*) indica, dentre as instâncias verdadeiramente positivas, qual proporção foi corretamente identificada pelo modelo. Uma revocação alta significa poucos falsos negativos.

F1-Score é a média harmônica entre precisão e revocação, sendo a métrica mais adequada para problemas com desbalanceamento de classes, pois penaliza modelos que sacrificam uma métrica em favor da outra.

ROC-AUC mede a área sob a curva ROC, que relaciona a taxa de verdadeiros positivos à taxa de falsos positivos em diferentes limiares de decisão. Um valor próximo de 1.0 indica que o modelo consegue distinguir bem entre as classes, independentemente do limiar escolhido.

Além das métricas agregadas, foram geradas as matrizes de confusão de ambos os modelos, permitindo uma análise detalhada dos erros cometidos — em particular, a proporção de falsos positivos e falsos negativos — e o gráfico de importância de features do Random Forest, que evidencia quais atributos mais contribuíram para as decisões do modelo.

8 Resultados

Nesta seção são apresentados os resultados obtidos pelos modelos treinados para o problema de correlação entre usuários e repositórios do GitHub.

Foram avaliados os modelos *Random Forest* e *Logistic Regression*, utilizando métricas de classificação binária como Accuracy, Precision, Recall, F1-Score e ROC-AUC.

8.1 Comparação dos Modelos

A Figura 1 apresenta a comparação entre os modelos considerando a métrica F1-Score.

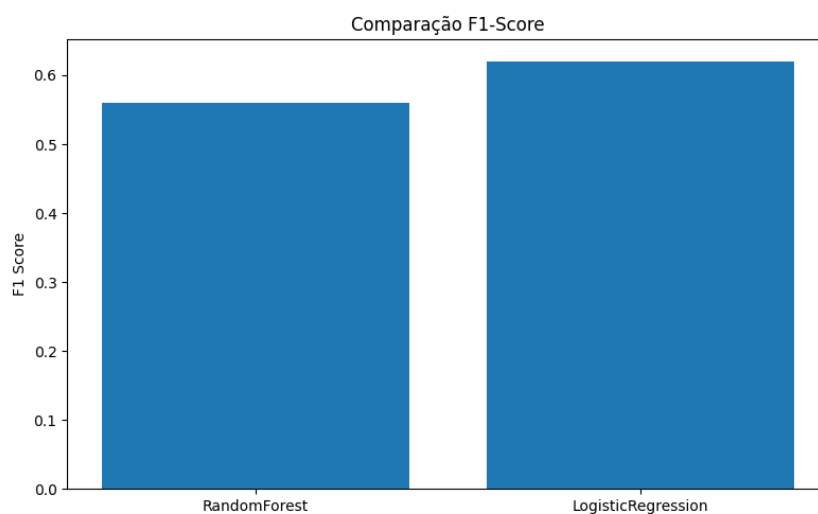


Figura 1: Comparação de F1-Score entre os modelos.

Observa-se que o modelo Logistic Regression apresentou melhor equilíbrio entre precisão e recall, obtendo o maior valor de F1-Score.

A Figura 2 apresenta a comparação utilizando a métrica ROC-AUC.

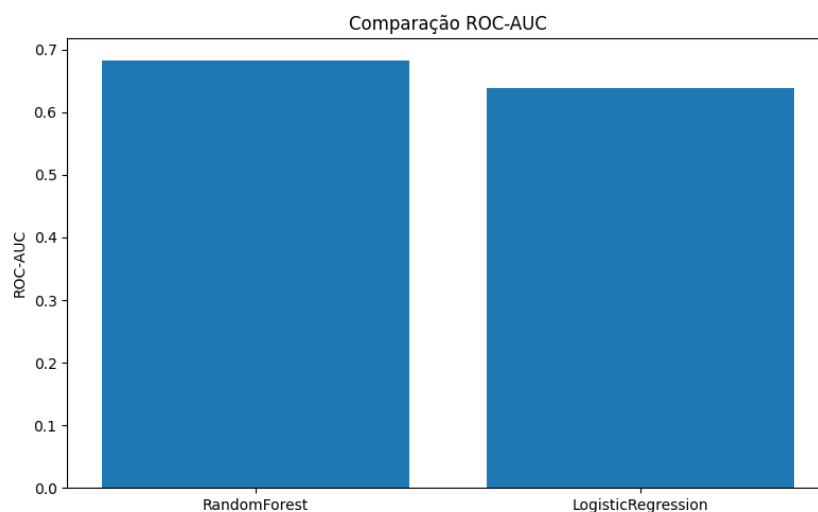


Figura 2: Comparação da métrica ROC-AUC entre os modelos.

Nesse cenário, o modelo Random Forest apresentou melhor capacidade de separação entre as classes, obtendo maior valor de ROC-AUC.

8.2 Matrizes de Confusão

As matrizes de confusão permitem analisar a distribuição de acertos e erros realizados pelos modelos.

A Figura 3 apresenta a matriz de confusão do modelo Random Forest.

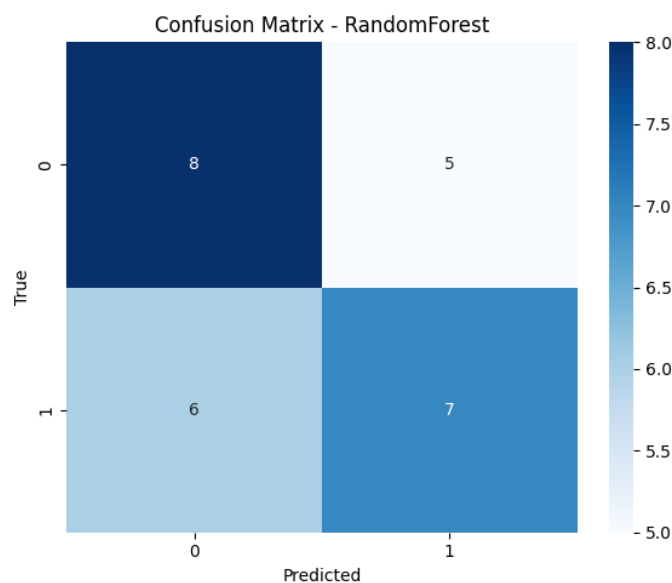


Figura 3: Matriz de confusão do modelo Random Forest.

A Figura 4 apresenta a matriz de confusão do modelo Logistic Regression.

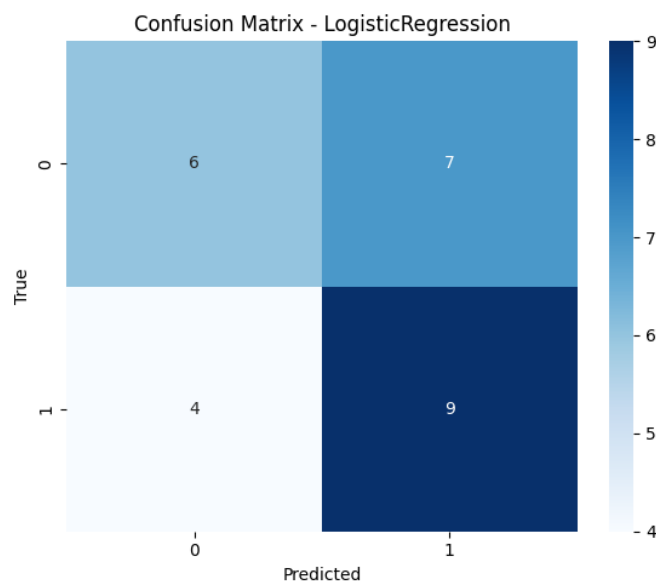


Figura 4: Matriz de confusão do modelo Logistic Regression.

Pode-se observar que os modelos apresentam comportamentos distintos em relação aos falsos positivos e falsos negativos, refletindo diferenças entre abordagens lineares e baseadas em árvores de decisão.

8.3 Importância das Features

A Figura 5 apresenta a importância das features calculada pelo modelo Random Forest.

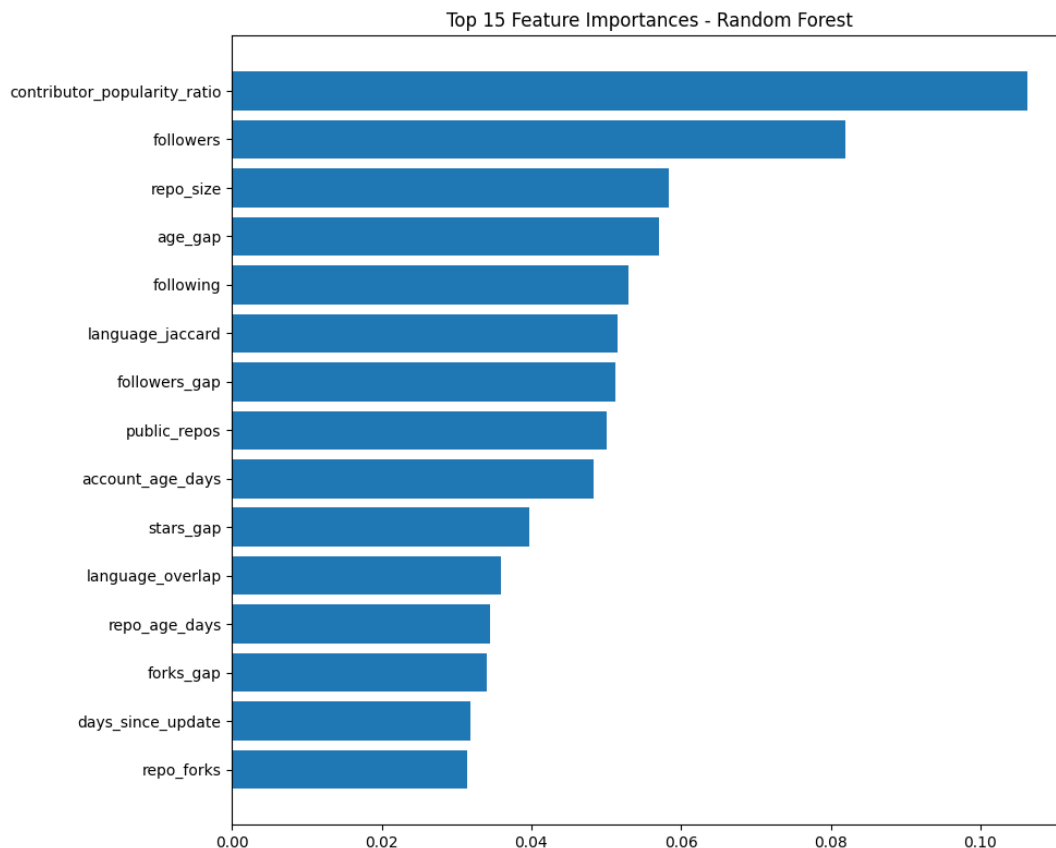


Figura 5: Importância das features no modelo Random Forest.

As features relacionais associadas à similaridade entre usuários e repositórios apresentaram relevância significativa para o processo de classificação, indicando que informações de compatibilidade entre linguagens, atividade e popularidade contribuem diretamente para o desempenho do modelo.

Tabela 11: Comparação dos modelos utilizados

Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.5769	0.5833	0.5385	0.5600	0.6834
Logistic Regression	0.5769	0.5625	0.6923	0.6207	0.6391

8.4 Análise dos Resultados

Os resultados obtidos pelos dois modelos avaliados são apresentados na Tabela 11. De forma geral, ambos os modelos alcançaram acurácia idêntica de 57,69%, o que indica que, em termos

de acertos totais, nenhum dos dois se destacou. No entanto, as diferenças tornam-se evidentes ao analisar as demais métricas individualmente.

O modelo *Random Forest* obteve maior precisão (0,5833 contra 0,5625), significando que, dentre os pares classificados como contribuidores, uma proporção ligeiramente maior era de fato positiva. Por outro lado, sua revocação foi inferior (0,5385 contra 0,6923), indicando que o modelo deixou de identificar uma parcela relevante dos contribuidores reais — ou seja, cometeu mais falsos negativos. Em contrapartida, o *Random Forest* apresentou o maior valor de ROC-AUC (0,6834), sugerindo que sua capacidade de separação probabilística entre as classes é superior, independentemente do limiar de decisão adotado.

A *Regressão Logística*, por sua vez, obteve o maior F1-Score (0,6207), resultado direto de seu melhor equilíbrio entre precisão e revocação. Seu valor elevado de recall (0,6923) indica que o modelo é mais conservador em rejeitar pares positivos, priorizando a identificação de contribuidores mesmo ao custo de mais falsos positivos. Essa característica pode ser desejável em cenários onde deixar de identificar um contribuidor real é mais custoso do que uma classificação incorreta de não-contribuidor.

Os valores moderados das métricas em ambos os modelos eram esperados, dado o grau de dificuldade inerente ao problema. O dataset foi construído com *negativos difíceis* — pares em que o usuário compartilha linguagens com o repositório, mas não contribui — o que torna a tarefa de discriminação mais desafiadora e aproxima o problema de cenários reais. Além disso, o tamanho reduzido do conjunto de dados, consequência das limitações de taxa da API do GitHub, pode ter impactado a capacidade de generalização dos modelos.

É importante destacar que os valores de ROC-AUC acima de 0,63 para ambos os modelos indicam que há informação discriminativa real nos atributos coletados, superando um classificador aleatório (ROC-AUC = 0,5). Isso valida a relevância das features construídas — em especial as relacionais, como compatibilidade de linguagens, diferenças de popularidade e métricas de atividade — para a predição de contribuição em repositórios GitHub.

8.5 Distribuição das Classes e Correlações

A Figura 6 apresenta a distribuição das classes no dataset final, evidenciando o balanceamento aplicado entre pares positivos e negativos.

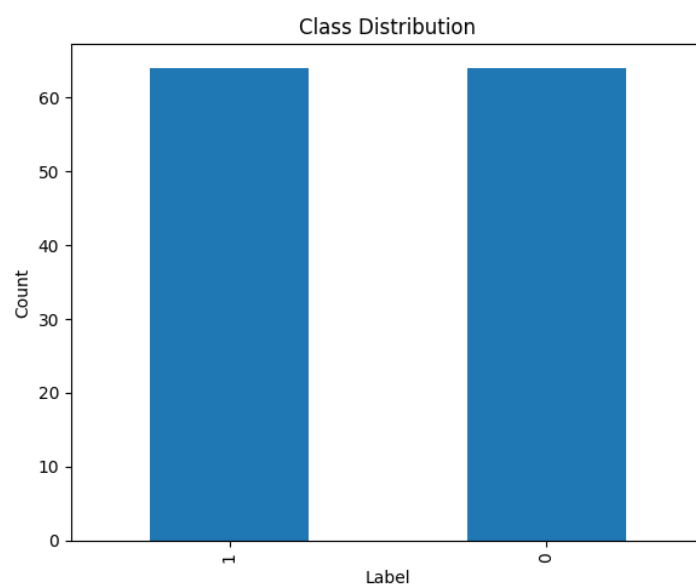


Figura 6: Distribuição das classes no dataset relacional.

A Figura 7 apresenta as correlações entre as features e a variável alvo (*label*), permitindo identificar quais atributos possuem maior relação linear com a contribuição do usuário ao repositório.

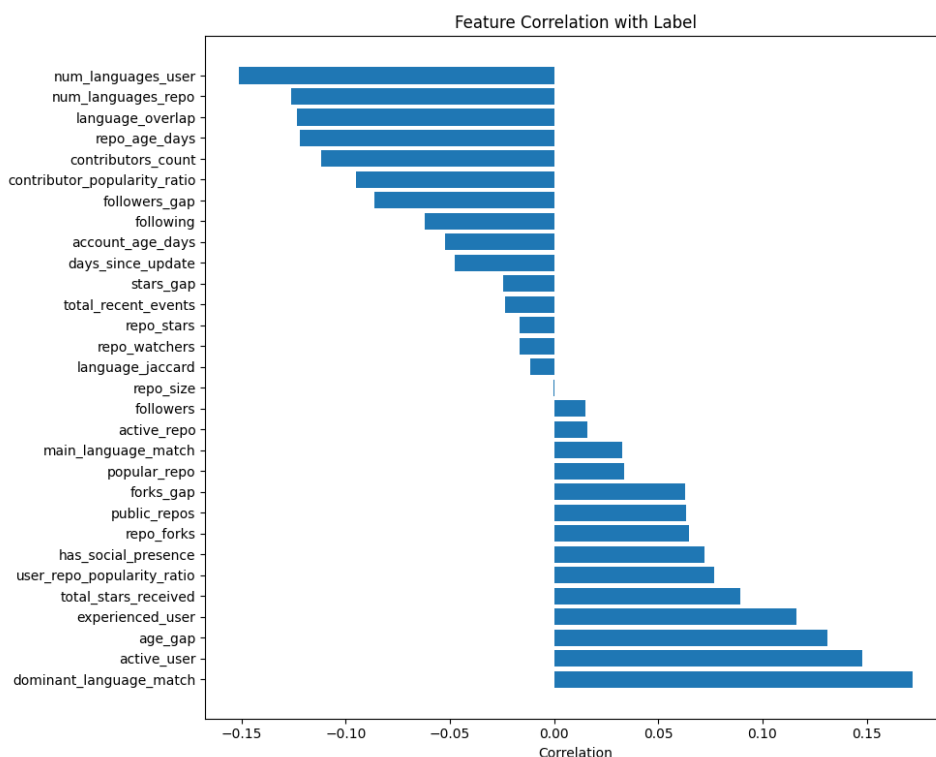


Figura 7: Correlação das features com a variável alvo.

A Figura 8 apresenta a matriz de correlação entre todas as features numéricas do dataset relacional, evidenciando relações de redundância e colinearidade entre os atributos.

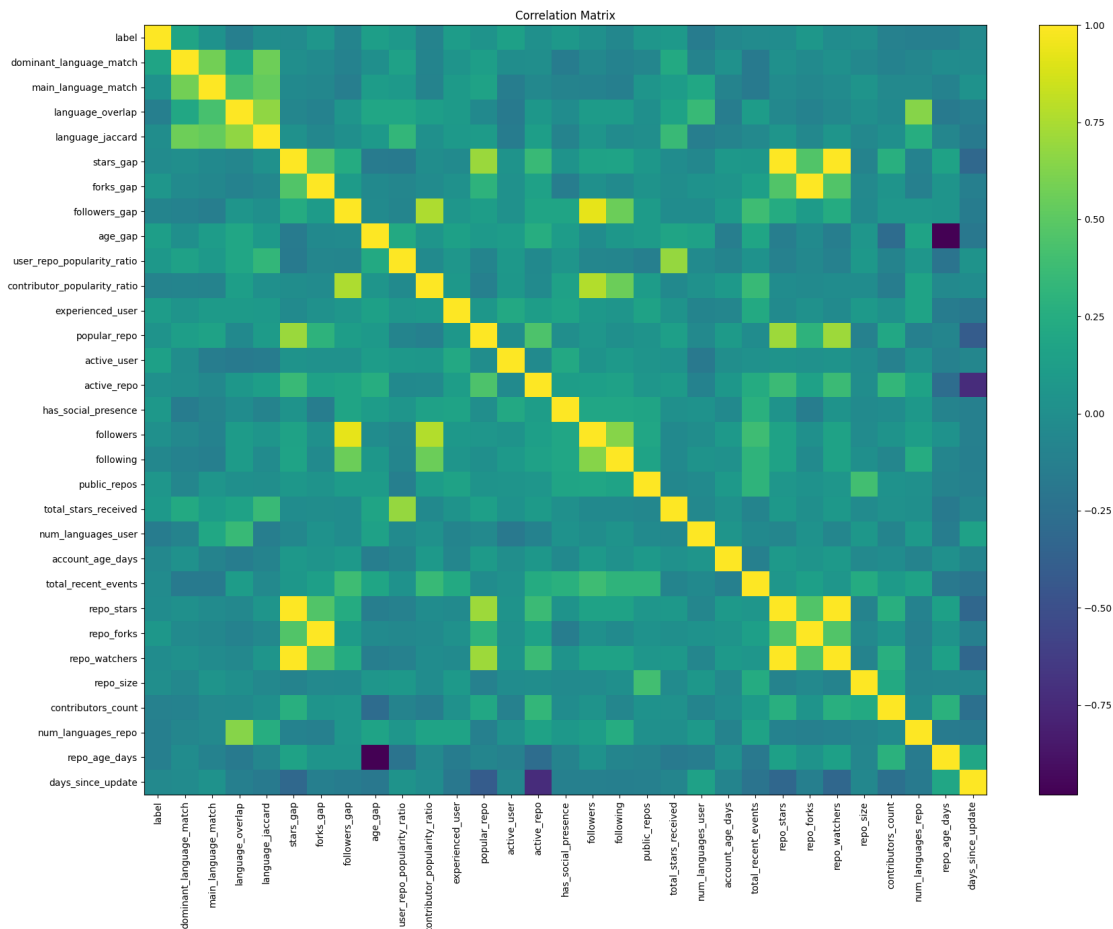


Figura 8: Matriz de correlação entre as features do dataset relacional.

Observa-se que features como `repo_stars`, `repo_forks` e `repo_watchers` apresentam alta correlação entre si, o que era esperado dado que todas medem aspectos relacionados à popularidade do repositório. Da mesma forma, as features de eventos do usuário tendem a se correlacionar, refletindo um padrão geral de atividade na plataforma. Essa redundância reforça a importância da etapa de seleção de features para evitar que atributos colineares influem artificialmente a importância de determinadas dimensões no modelo.